# Security Relevant Changes in Reused Agent Skills

CSC 580 Advanced Software Engineering
University of Michigan-Flint, Fall 2026
Group 8 Final Project Report

Jim Prantzalos Kyle Cantrell Vaisnavii Mohanraj Christian Heiney

November 8, 2026

## Abstract

AI coding agents can use reusable instruction packages, called *agent skills*, to carry out specialized tasks. Reusing a skill saves effort but can also introduce changes to commands, scripts, external dependencies, and other security-sensitive instructions. GitSkills is a large collection of agent-skill files found in public GitHub repositories. Its many repeated and related skills make it useful for studying changes associated with reuse. We investigated whether related skills in this dataset contain security-sensitive behavior absent from an earlier observed artifact. Our static-analysis pipeline identifies related skill content, infers plausible comparison direction from available evidence, compares changed operations, and evaluates those changes against eleven defined risk indicators. The indicators identify changes worth reviewing, not confirmed vulnerabilities.

We analyzed **three non-overlapping, stratified random samples totaling 300 skill families**, using 6,570 artifacts and 2,157 selected primary directed comparisons. The automated scanner identified supported indicators in **63 families (21.0%)** and **340 comparisons (15.8%)**. Local script execution and external code-acquisition changes appeared across the most families. Comparison-level rates varied much more across trials, largely because a few families contributed many related comparisons; a sensitivity calculation excluding each trial's largest contributor reduced the combined pair-level rate to 8.4%, while the family-level rate remained 20.2%. These sample percentages do not estimate prevalence across all GitSkills. Some comparison directions and supporting-file content remain uncertain, and independent human validation is in progress. The study provides reproducible evidence of candidate security-sensitive changes while distinguishing observed differences from verified historical or security outcomes.

## Introduction

Adopting a familiar agent skill can introduce unfamiliar instructions. A reused file may preserve its name and most of its text while adding a downloader, changing a remote code source, or expanding a file operation. Reviewing only the skill description, the presence of a capability, or the total number of scanner matches can miss the change that matters to the adopter.

The Agent Skills format combines a Markdown instruction file with optional scripts, references, and assets (Agent Skills, n.d.). That structure makes the file body and its associated resources relevant to a review. A script can change without changing the instruction that invokes it. Conversely, a new example or additional documentation can increase matches without introducing a new operational requirement. The security question is therefore comparative and contextual: what changed between related artifacts, and does the change satisfy an explicit reason for review?

The study addresses that question through static analysis of the GitSkills dataset (Destefanis et al. 2026). The pipeline separates three decisions: whether two artifacts are meaningfully related, whether an earlier/source-to-later ordering is defensible, and whether changed evidence supports one of eleven risk indicators. A URL reference, a rule-count increase, and a supported risk indicator are distinct observations. None alone establishes malicious intent or execution.

The report presents the comparison method, eleven indicators, exploratory results, and validation requirements. Appendices retain preparation queries, implementation details, validation materials, extended results, and supporting repository resources.

## Research question and study scope

> **Do modified or reused skills introduce command execution, file-system access, network access, or other risky behavior that was absent from an earlier version or source artifact?**

The study operationalizes this question as a comparison of stored instructions between related artifacts. An **artifact occurrence** is one skill at a repository and path in the dataset. A **skill variant** groups identical file content, while a **bundle variant** also accounts for the known state of sibling resources. A **candidate family** contains qualifying occurrences with the same exact name. Sharing a name defines a search space; it is not evidence of reuse by itself.

The unit of directional analysis is an ordered pair of related artifacts. Its endpoints are designated **base** and **derived** according to the available direction evidence. These terms identify the comparison orientation, not proven original authorship or direct copying. Equivalent and direction-unknown relationships are retained separately rather than treated as negative security findings.

The primary reported outcome is the **proportion of sampled families** with at least one directed comparison supporting a defined risk indicator. The pair-level outcome is secondary, because one family can contribute many related comparisons. The indicator assesses a particular introduced or changed operation, not merely a new keyword, a larger file, or a higher count of matches. For example, changing the source of an executed remote script can matter even if both artifacts contain exactly one download-and-execute command. Replacing a package manager for an otherwise equivalent installation need not introduce a new dependency-acquisition operation.

The group's expectation is that some related artifacts will contain observable security-sensitive changes, but that many differences will reflect routine adaptation, packaging, or legitimate functionality. The competing explanation is that a reported indicator identifies an intended feature rather than unwanted exposure. The study does not measure a causal effect of reuse, execute instructions, or attempt comprehensive security detection. The eleven indicators define the scope of the assessment; behavior outside that scope is not automatically a scanner error.

## Background and related work

GitSkills supplies the empirical basis for studying skill occurrences and their copies. Its release retains repository/path identities alongside content hashes, with enrichment for representative content (Destefanis et al. 2026). That distinction motivates the separation of occurrence-level comparisons from distinct content and bundle states. Many comparisons can

reuse the same content, so comparison counts cannot be interpreted as independent historical modification events.

The Agent Skills specification defines the instruction file and optional resource organization (Agent Skills, n.d.). It describes a format; it does not establish that a particular skill is trustworthy or that its instructions executed. The assessment uses this structure to inspect available bodies and associated files without execution.

The project's specific contribution is the combination of relatedness, recorded direction evidence, construct-level comparison, and an explicit indicator assessment. The supply-chain concern is the opportunity for changed instructions or dependencies to enter an adopted artifact. The method tests for bounded static signals of that change. It does not classify all artifacts as safe or unsafe, reconstruct complete copying histories, or replace runtime investigation. The scope also distinguishes the present results from the earlier Sprint 1 integration checks, which primarily exercised rule-count comparisons.

## Dataset and cohort

### Data source and unit of analysis

We use GitSkills version 1.0.0, released on Zenodo on August 10, 2026. GitSkills collected agent-skill files from public GitHub repositories in July 2026. The release contains 3,797,117 **file occurrences**, including repeated copies of the same content (Destefanis et al. 2026). A skill generally consists of a `SKILL.md` instruction file and may include supporting files—called **sibling files** in the dataset—such as scripts or reference material. We import the released SQLite data into a local DuckDB database for analysis.

Our starting unit is a **skill family**: qualifying artifacts with the same exact skill name, even when their descriptions differ. The shared name identifies candidates for comparison; it does not establish that they are copies of the same skill. Within a family, the similarity analyzer groups sufficiently related skill bodies into **clusters**. We then examine selected **directed comparisons** between related artifacts for which the available evidence supports a plausible earlier-to-later order. The relationship and direction methods are explained in the next section.

### Preparing comparison candidates

The project builds an `artifact_groupings` table from the GitSkills records. An artifact qualifies when its **frontmatter** (the YAML metadata at the start of `SKILL.md`) parses as valid, its instruction body is longer than 50 characters, and its content, name, and description are present. A candidate family must have at least two qualifying occurrences with the same name. Descriptions are normalized for supporting subgroups, but differences in description do not split a family during similarity analysis.

An earlier preparation audit retained 943,257 occurrences in 162,995 same-name families. These are **candidate-pool counts**, not the denominator for our study and not the number of families that passed the final sampling checks. Appendix A provides the preparation SQL and audit. The saved Sprint 3 sampling reports record the candidate pools used when each trial was drawn.

| Family size (artifact occurrences) | Per trial | Three trials |
|---|---|---|
| Low: 2 | 20 | 60 |
| Medium: 3–15 | 55 | 165 |
| Large: 16–100 | 20 | 60 |
| X-Large: 101–500 | 4 | 12 |
| Extreme: 501 or more | 1 | 3 |
| **Total** | **100** | **300** |

Table 1: Family-size strata and allocation across the three trials

### Selecting the three Sprint 3 trials

We analyzed **three non-overlapping, stratified random samples of 100 families each**. This is the final study population for the report; earlier Sprint 1 and Sprint 2 cohorts helped develop and test the method but do not contribute to the Sprint 3 results.

A family enters the random selection process only if it passes two checks (RDR-014):

1. **Distinct states:** At least two combinations of the main skill-file **hash** (an identifier derived from its exact content) and **sibling fingerprint** (a summary of known supporting-file contents) differ. Repeated copies of the same known state do not meet this requirement.
2. **Usable relationship:** The similarity analyzer can identify at least one primary directed comparison involving different main-file hashes or different *known* sibling hashes. A different hash alone does not establish relatedness or direction.

When a randomly drawn candidate fails the relationship check, the picker records the rejection and draws a replacement from the same size group. **Selection does not run the security scanner** and does not favor families with risk indicators. Families that fail eligibility are excluded from the study population, not counted as families with no risk.

We divided candidate families into five groups by their **total number of artifact occurrences**, including repeated copies. The allocation deliberately gives larger families more representation than they have in the candidate pool, so the study includes both ordinary and unusually large families without letting the largest ones consume the entire sample.

The trials use recorded random seeds **5801**, **5802**, and **5803**. Trial 2 excludes names selected in Trial 1; Trial 3 excludes names selected in both previous trials. Each trial saves its `families.txt` and `sampling_report.json`, preserving the selection and rejection record. These are separate, non-overlapping samples, but the exclusions mean they are not independent draws from an unchanged pool.

The allocation is **not proportional** to the wider candidate population. Our unweighted percentages therefore describe these 300 selected, relationship-eligible families; they must not be presented as population-wide estimates for GitSkills. The number of eligible families in each stratum after relationship screening has not been established for population weighting.

### What we analyzed

The three trials loaded **6,570 artifacts** with none skipped. Their similarity analysis selected **2,157 primary directed compar-**

**isons** in total. A *primary* comparison connects a derived candidate to at most one selected predecessor, using the graph method described in Section 6. We did not scan additional related directed edges in these trials, because doing so could count overlapping changes repeatedly within the same family (RDR-013). Equivalent or direction-unknown relationships are retained for relationship review but are not assigned introduced/removed security behavior.

The main outcome is **family-level frequency**: how many sampled families contain at least one comparison with a supported risk indicator. We also report pair-level frequency using all analyzed primary directed comparisons as its denominator. The two measures answer different questions: how widely a signal appears across families versus how frequently it appears among selected comparisons. Comparisons within a family may share artifacts and are not independent historical events.

### Evidence availability and safe handling

The main skill bodies were available for all loaded artifacts, but commit dates and supporting-file content are not always available. A known empty sibling bundle is different from one whose contents could not be retrieved. We report unavailable files and partial evidence extraction rather than treating them as proof that no behavior exists.

All skills and bundled files are untrusted data. We do not execute their commands or scripts, install referenced dependencies, fetch URLs, or clone repositories to replace missing source content. The saved manifests identify the dataset, rules, code revision, and selected inputs; local numeric artifact IDs alone are not portable across database builds. The full data dictionary and reproduction paths are linked in Appendix E.

## Method

### Research approach and security scope

Our question asks whether related skills introduce security-sensitive behavior absent from an earlier observed form. GitSkills contains skill-file snapshots and some repository and commit metadata, but **not a complete version history or a verified record of which skill was copied from which**. Some commit dates are missing; a first commit at the current file path may also be later than the skill's true origin. We therefore infer plausible relationships and ordering before comparing behavior. We do not claim to reconstruct a verified family tree.

Our threat model focuses on observable instructions or code involving command execution, network access, file-system operations, credentials, external code or dependency acquisition, and system modification. These operations can be legitimate. They matter because an agent acting on untrusted instructions may have tools that affect a user's machine, data, or external services. A detected change is a **reason to review**, not evidence that an instruction ran, a vulnerability was exploited, or its author intended harm. The full threat model describes the trust boundary and coverage limits.

### Step 1: Identify related skills

We first compare artifacts within an exact-name candidate family (RDR-003). Identical `SKILL.md` content is recognized by its file hash; known bundled-file content is represented separately by a fingerprint of sibling file hashes. This matters because two copies of `SKILL.md` can contain the same instructions yet ship with different scripts. Unavailable sibling state is treated as unknown, not as an empty bundle.

| Priority | Direction evidence | What it establishes—and what it does not |
|---|---|---|
| 1 | **Declared source** | Supported provenance metadata identifies a source; declarations can still be wrong or incomplete. |
| 2 | **Direct chronology** | `first_commit_at` records when a state was observed at its current file path, not its true original creation date. |
| 3 | **Equivalent-peer chronology** | A different occurrence of the *same known main-file and sibling state* supplies an earlier observed date when direct history is missing or later. The date belongs to the state, not automatically to the undated occurrence. |
| 4 | **Repository-creation boundary** | If one state was observed before another repository existed, that other repository occurrence could not have been earlier. Repository age alone never dates a skill. |
| 5 | **Directional containment** | When stronger evidence is missing, substantially greater containment in one direction (difference of at least **0.10**) can suggest a plausible base-to-expanded order. This is the weakest direction signal. |

Table 2: Evidence used to infer earlier-to-later ordering, in priority order

For distinct main-file contents, we compare the body text after masking the opening YAML frontmatter. We normalize and tokenize the text, then form **five-token shingles**: sequences of five consecutive tokens. Comparing sets of these short sequences tolerates some formatting changes while retaining important command and identifier structure (RDR-004).

We require three signals of relatedness: at least **five shared shingles**, maximum **containment of 0.80**, and **Jaccard similarity of 0.30**. Containment asks how much of the smaller shingle set appears in the larger one; Jaccard measures the shared shingles relative to all distinct shingles across both texts. For example, if two files have 100 and 150 shingles, with 85 shared, containment is `85 / 100 = 0.85` and Jaccard is `85 / 165 = 0.515` (approximately 0.52). The first captures retained content in an expanded skill; the second guards against treating a small shared fragment as broadly similar. Using both is useful because the *added* material in a modified skill may contain the behavior we seek.

The analyzer groups related artifacts into **similarity clusters**. An exact shared name is therefore only a starting point: a large family can contain several clusters of differently written skills. The thresholds are documented operational choices, not probabilities of copying.

### Step 2: Infer which artifact could come first

We assign an earlier (**base**) and later (**derived candidate**) only after establishing relatedness. The available evidence is tested in the following order (RDR-007):

For example, if a skill state has no recorded commit date but an equivalent copy was observed in January, that peer establishes that the state existed by January. If another related state is observed in March, January may support an earlier/later comparison. However, we cannot transfer that peer's date when its bundled-file state is different or unknown. Likewise, a repository created in March does **not** prove that its skill is newer than a skill in an older repository; a boundary is useful only when an independently dated state predates the repository.

When none of these methods supports direction, the relationship remains unresolved. Equivalent states remain peers rather than being forced into an earlier/later order. Each directed comparison retains its direction basis so that findings based on containment alone can be interpreted more cautiously than those supported by observed chronology.

The algorithm selects at most **one primary predecessor per derived candidate**, avoiding cycles. It can also identify other related, directed comparisons, called *additional edges*. RDR-013 selected **primary edges only** for the final three trials to study more families without repeatedly counting all related combinations in a few large families. In a controlled Sprint 2 comparison using the same scanner configuration, primary selection reduced 3,997 all-edge comparisons to 468 while retaining 43 of 44 observed family–indicator combinations. This supported the narrower design but did not establish that additional edges never contain unique findings. No separate Sprint 3 all-edge robustness result is claimed here.

### Step 3: Detect and compare observable evidence

The static scanner applies **15 detection rules across six categories**: command execution, network access, file-system access, credentials, external code/dependency acquisition, and system modification. The rules identify recognizable constructs, such as a shell command, URL, file operation, or package-install instruction. They do not execute the content. The `SKILL.md` frontmatter is excluded from behavioral scanning because descriptive metadata and declared tools do not, by themselves, demonstrate an operation. Provenance metadata may still inform Step 2 (RDR-002).

We then **extract and compare evidence**, rather than interpreting a changed rule-match count as new risk. Evidence records retain the recognized operation, source location, context, and extraction completeness. The comparison distinguishes added, removed, and matching constructs. A different URL might matter even if the number of URL matches stays the same; repeated mentions can raise match counts without introducing an operation. Unclear associations are left uncertain (RDR-008 and RDR-012).

Bundled **sibling files** are assessed when added or changed content is available, regardless of file extension. For instance, a skill's instructions might remain the same while the bundled `setup.sh` changes. We examine the changed script's stored content, but do not infer its behavior from a filename when that content is missing. Known identical-content renames do not count as changed script behavior (RDR-009).

The comparator also distinguishes `skill-only`, `siblings-only`, `skill+siblings`, and `unknown` bundle changes. Here, **equivalent skill content** can mean equivalent under the token-shingle representation, not necessarily byte-for-byte identical; therefore, a `siblings-only` label does not by itself guarantee that no textual differences exist. Appendix B defines the classifications.

| ID | Security-sensitive change assessed |
|---|---|
| **RI-001** | Newly downloaded content is passed directly to an interpreter. |
| **RI-002** | The source URL of remotely executed code changes. |
| **RI-003** | A network operation uses a new hostname/domain or introduces explicit outbound transfer. |
| **RI-004** | HTTPS is downgraded to HTTP, or certificate verification is disabled. |
| **RI-005** | External code or dependency acquisition is introduced, or its source/registry changes. Package-name or version edits alone do not qualify. |
| **RI-006** | A destructive file operation is introduced or expands in scope or privilege. |
| **RI-007** | An operation newly accesses a recognized credential file or includes a credential reference directly in outbound transfer. |
| **RI-008** | An operation introduces or expands system, permission, service, or scheduled-task changes. |
| **RI-009** | A read/write operation newly targets a recognized sensitive file or directory. |
| **RI-010** | Local script execution is introduced or changes to a target with observably different content. |
| **RI-011** | A skill newly directs following external instructions, changes that source, or changes an ordinary reference into such a directive. |

Table 3: Eleven evidence-based risk indicators

### Step 4: Assess changes against risk indicators

The last stage tests the changed evidence against **11 risk indicators (RIs)** selected in RDR-010. Their purpose is to highlight observable ways in which an operation or source may change the agent's exposure to external code, networks, local files, credentials, or privileged actions. They do **not** assign a severity score. An indicator must be supported by the relevant *change* and its context; a rule match alone is insufficient.

We chose these indicators because they can be supported by **specific observable evidence** in the dataset rather than speculative runtime effects. They also reflect the plausible changes described in the threat model. An indicator may be supported, uncertain, or absent from the observed comparison. "Absent" means **not supported by available evidence**, not proven safe. One command may support several indicators without representing several separate incidents.

A useful negative example from scanner development is a skill that changes `npm install` to `bun install` while retaining the same dependency-acquisition workflow. We still record the changed commands, but the substitution alone does not support RI-005. This distinction avoids inflating counts whenever routine instructions change (RDR-012). In contrast, introducing `curl -fsSL https://cli.inference.sh | sh && infsh login` adds an explicit download-and-execute operation. The Trial 3 `python-executor` comparison **1773238 → 1729578** received RI-001 and RI-003 for this single changed construct; its classification remains an **automated observation**

**pending human adjudication**. Two indicators here describe two aspects of one command, not two incidents.

### Outcomes and interpretation limits

Our **primary measure** is the number of sampled families with at least one supported RI divided by all sampled families. Secondary measures count primary directed comparisons containing at least one RI, the distinct families containing each indicator, and the distribution by family size, direction basis, and evidence coverage. Indicator frequencies can overlap, so their totals are not additive.

A **flagged comparison** may contain rule changes, extracted-evidence changes, or uncertain observations even if no RI is supported. A **supported finding** meets a defined indicator's automated criteria; it is not a validated security incident. We separately report partial extraction, missing sibling content, and uncertainty. Complete coverage means no known gap *within the implemented extraction scope*, not comprehensive detection of all risks. Appendix B contains the rule registry and analysis workflow; Appendix E identifies the source artifacts and saved outputs used for these results.

## Results

### Three-trial overview

The saved Sprint 3 analysis covers **300 distinct, randomly selected families** in three equal-sized, non-overlapping samples. The pipeline loaded **6,570 artifacts** and analyzed **2,157 primary directed comparisons**. It automatically identified at least one **supported risk indicator (RI)** in **63 families (21.0%)** and **340 comparisons (15.8%)**. These are observations under the study's implemented rules, **not** manually confirmed vulnerabilities, malicious skills, or actual security incidents. Independent review of the selected validation cases is still in progress.

Family percentages use 100 families per trial (300 combined); pair percentages use all primary directed comparisons in the relevant trial. No selected artifact was skipped. Each trial used the same code revision, rule registry, similarity policy, and primary-edge setting. The 300 families are the study sample, not a representative cross-section of every GitSkills skill.

### Finding 1: Detections occur across several families

The primary outcome—families containing at least one supported RI—ranges from **18% to 25%** across trials. This demonstrates that the scanner's indicators are not limited to one family or one trial. It does not yet establish the proportion of all GitSkills families with such changes, or how many observed detections independent reviewers will confirm.

The family-level outcome is intentional (RDR-013): a family contributes **one** to this measure whether one of its comparisons or many of its comparisons contain an indicator. The pair-level measure provides a useful second view, but answers a different question.

### Finding 2: Pair-level totals are sensitive to a few families

As the figure shows, the comparison-level rate varies more than the family-level rate. A reason is that some families yield far more primary comparisons than others. Within a family, multiple comparisons can share artifacts and recurring evidence; they should not be counted as independent historical changes.

The most influential family in each trial contributed the following indicator-bearing comparisons:

To test the sensitivity of the descriptive result, we recalculated each trial's pair-level rate **without its most influential family**, leaving the original study sample and results untouched:

After excluding these three families, the family-level rate is **60 of 297 (20.2%)**, compared with **63 of 300 (21.0%)** originally. This contrast explains why family-level frequency is the primary measure. It also cautions against interpreting the 2,157 comparisons as independent security events.

### Finding 3: Indicators differ in how widely they appear

Risk indicators describe specific security-sensitive *changes*, not general scanner matches. The table counts the number of distinct sampled families containing each indicator and, secondarily, the number of directed comparisons where it was supported. A family or comparison can appear in several rows.

RI-010 is observed across the most families, followed by RI-005. By contrast, RI-006 produces almost as many comparison-level observations as RI-005 but is found in only **10 families**. Of RI-006's 159 indicator-bearing comparisons, **134** occur in `cso` and `mcp-builder` together. The difference between these columns is important: a high pair count can indicate repeated comparisons within a few families rather than wide distribution across the sample.

RI-002, RI-004, and RI-009 were not supported in these trials. Zero detections could reflect the sampled content, missing evidence, or the implemented rules. It does not establish that those behaviors are absent from GitSkills.

### Supporting observation: Family size and cluster structure

Larger families were more likely to contain at least one supported RI. They also offered more candidate relationships and, often, more **similarity clusters**—groups of artifacts whose instructions are related under the study's similarity criteria.

For example, `implement` contains **856 artifacts in 35 clusters**, whereas `cso` contains **150 artifacts in two clusters**. Many occurrences under one name do not necessarily represent a single shared history: different skills may use the same name. Conversely, several clusters do not prove different authors or independent origins. The three Extreme families are too few to support a broad conclusion about their 100% detection rate.

These patterns also help explain why the study starts with same-name candidates but requires content similarity before choosing comparisons. They do **not** show that adding copies or increasing family size causes security risk.

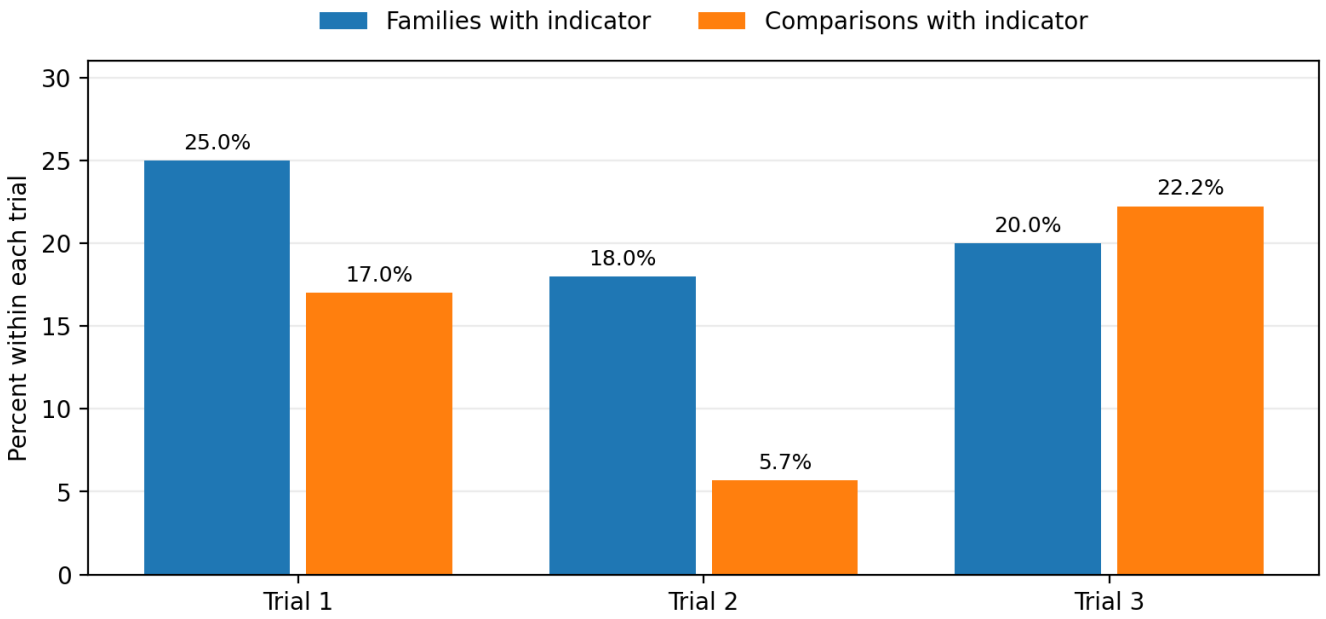



Figure 1: Family-level versus comparison-level rates in the three trials. Family-level rates are more similar across trials.

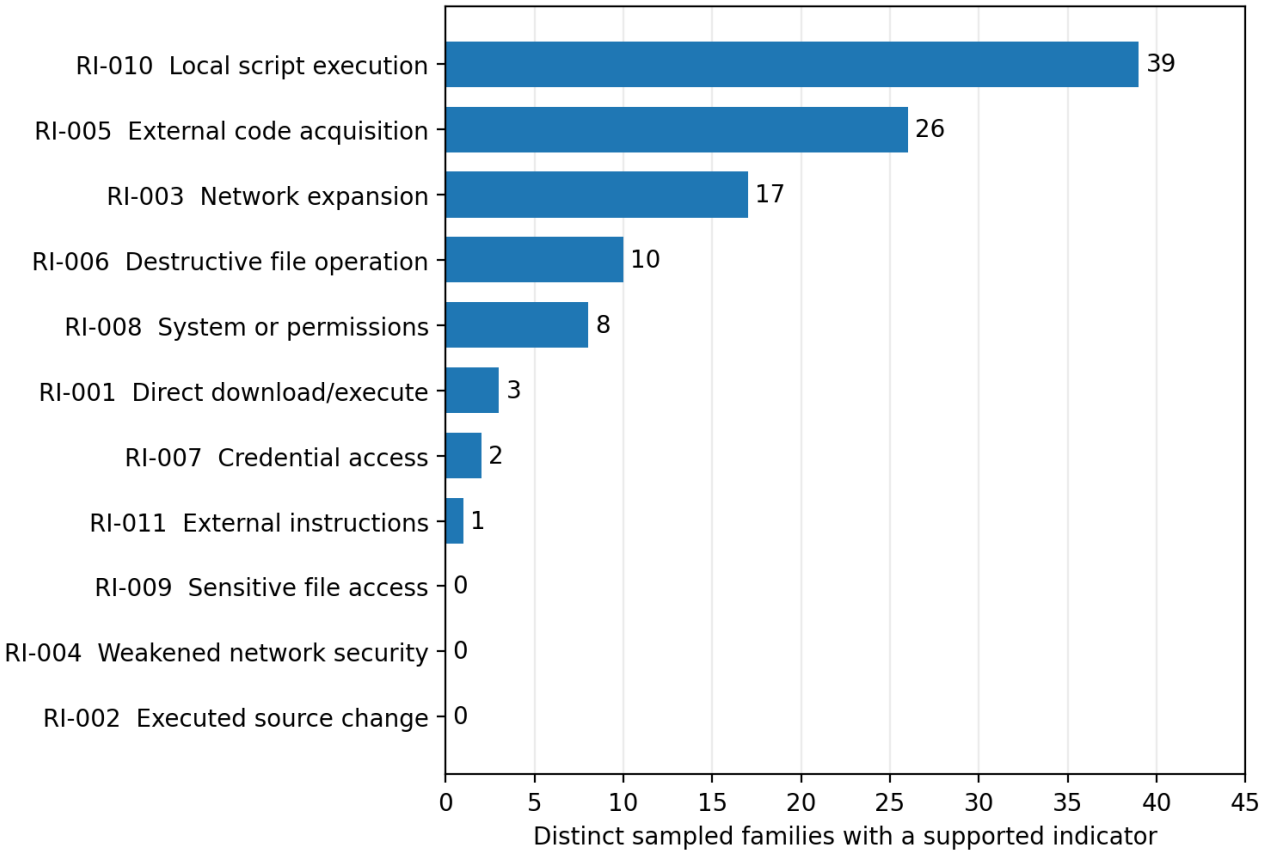



Figure 2: Number of distinct families with each supported risk indicator; indicators with no detections are included at zero.

| Measure | Trial 1 | Trial 2 | Trial 3 | Combined |
|---|---|---|---|---|
| Families analyzed | 100 | 100 | 100 | **300** |
| Artifacts loaded | 2,275 | 2,067 | 2,228 | **6,570** |
| Primary directed comparisons | 665 | 633 | 859 | **2,157** |
| Families with at least one supported RI | 25 (25.0%) | 18 (18.0%) | 20 (20.0%) | **63 (21.0%)** |
| Comparisons with at least one supported RI | 113 (17.0%) | 36 (5.7%) | 191 (22.2%) | **340 (15.8%)** |
| Flagged comparisons (any reason) | 346 | 271 | 502 | **1,119** |
| Comparisons with incomplete evidence coverage | 123 | 28 | 60 | **211** |

Table 4: Automated results from the three Sprint 3 trials

| Trial | Family | Indicator-bearing comparisons | Share of that trial's indicator-bearing comparisons |
|---|---|---|---|
| 1 | `cso` | 55 | 48.7% |
| 2 | `junta-leiloeiros` | 8 | 22.2% |
| 3 | `mcp-builder` | 133 | 69.6% |

Table 5: Most influential family in each trial

| Trial | All families | Excluding the listed dominant family |
|---|---|---|
| 1 | 113 / 665 = 17.0% | 58 / 549 = **10.6%** |
| 2 | 36 / 633 = 5.7% | 28 / 613 = **4.6%** |
| 3 | 191 / 859 = 22.2% | 58 / 560 = **10.4%** |
| **Combined** | **340 / 2,157 = 15.8%** | **144 / 1,722 = 8.4%** |

Table 6: Sensitivity of comparison-level detection rates to dominant families

| Indicator | Families (of 300) | Comparisons (of 2,157) |
|---|---|---|
| **RI-010** Local script execution | **39** | 201 |
| **RI-005** External code/dependency acquisition | **26** | 155 |
| **RI-003** Network expansion/redirection | **17** | 52 |
| **RI-006** Destructive/privileged file operation | **10** | 159 |
| **RI-008** System or permissions modification | 8 | 18 |
| **RI-001** Direct download-and-execute | 3 | 3 |
| **RI-007** Credential access/potential exposure | 2 | 4 |
| **RI-011** External instruction dependency | 1 | 1 |
| **RI-002**, **RI-004**, **RI-009** | **0 each** | **0 each** |

Table 7: Risk indicators by the number of sampled families and directed comparisons

### Supporting observation: Evidence changes are broader than supported indicators

The scanner flagged **1,119 of 2,157 comparisons (51.9%)** for some observed rule, evidence, or indicator-related change. Only **340 (15.8%)** satisfied the narrower requirements for at least one supported RI. Thus, **779 flagged comparisons** had no supported indicator. This separation is **by design**, not a claim that the other differences are harmless.

For example, a change from `npm install` to `bun install` can be correctly recorded as changed command evidence, yet it does not on its own demonstrate a newly introduced acquisition source (RI-005). The comparator retains the evidence; the assessor withholds an unsupported risk indicator. The contrasting `python-executor` case in Section 6 introduces an explicit remote-script execution method and a new network-operation hostname. These are examples of how assessment considers the *meaning of the change*, not merely the presence or number of matching words.

### Coverage and ordering qualifications

**Evidence coverage.** Of the **2,157 comparisons**, **211** have incomplete evidence coverage; **81** of the **340** comparisons with supported indicators are in this group. A specific operation can be fully extracted even when another portion of the same comparison is missing or partially extracted. The complete-labelled subset contains **259 indicator-bearing comparisons among 1,946 pairs (13.3%)**, compared with **340 among 2,157 (15.8%)** overall. This is a descriptive coverage check, not a correction for detection error or sampling bias.

**Direction evidence.** The 2,157 comparisons use declared source (10), direct chronology (399), equivalent-peer chronology (355), repository-creation boundaries (827), or directional containment (566). Of the 340 indicator-bearing comparisons, **131** rely on containment direction alone. Such relationships provide plausible orderings but not verified ancestry. Appendix D provides the full direction breakdown and the main change-type counts.

**Sibling differences.** The pipeline classifies **190 comparisons** as `siblings-only` under its bundle-equivalence rules, with

| Family size | Families | Families with an RI | Rate | Mean similarity clusters per family |
|---|---|---|---|---|
| Low (2 artifacts) | 60 | 4 | 6.7% | 1.0 |
| Medium (3–15) | 165 | 24 | 14.5% | 1.1 |
| Large (16–100) | 60 | 23 | 38.3% | 1.5 |
| X-Large (101–500) | 12 | 9 | 75.0% | 3.8 |
| Extreme (501+) | 3 | 3 | 100.0% | 21.7 |

Table 8: Family size, indicator frequency, and similarity-cluster structure

supported indicators in **41**. These observations support inspecting bundled resources, but most positive `siblings-only` comparisons are concentrated in `mcp-builder`; case-level review is needed before treating them as distinct changes in executed script behavior.

The independent manual reviews of the similarity decisions and the supported/unsupported indicators remain in progress. The results above are therefore final **automated trial counts** but provisional as claims about correctly detected security-sensitive changes. Their validation and the interpretation of individual cases belong in the evaluation and discussion sections.

## Evaluation and validation

### What the saved trials establish

The three Sprint 3 pipelines loaded all **6,570 selected artifacts**, produced auditable results for **2,157 primary directed comparisons**, and generated separate manual-review samples for each 100-family trial. Successful execution establishes that these outputs were produced under the recorded configuration; it does not establish that inferred relationships reflect historical copying, that every supported RI is correct, or that unflagged comparisons lack in-scope changes.

Automated tests and earlier constructed examples exercised the analysis logic, extraction, and output handling. Those checks are useful for regression testing, but they do not replace independent evaluation of the three final trials. The sample manifests, sampling reports, and validation inputs are retained separately to support reproducible review.

### Manual-review design

Each trial has a **30-case directed sample**: five each from the supported-indicator, uncertain-indicator, coverage-limited, sibling-change, flagged-without-indicator, and unflagged-directed strata. Each also has a **15-case relationship sample**: five ambiguous, five equivalent, and five no-related-edge cases. All three use manual-review seed **580**, producing **90 directed cases and 45 relationship cases** overall. These are review subsets of the existing trial results, not additional families or new trial data.

Directed reviewers inspect the original base and derived bodies, relevant stored sibling files, evidence correspondence, and source support for each RI. Relationship reviewers separately assess whether the similarity method establishes substantive relatedness and whether the available evidence supports the suggested order. This distinction matters: a detector may correctly identify a changed command while an inferred earlier/later relationship remains contestable.

Independent review precedes team adjudication. AI-assisted case notes, where used, are not independent human judgments. Reviewed annotations must match the frozen identities and checksums in the saved sample; a changed family list or regenerated review sample cannot silently substitute for an earlier one.

### Accuracy, uncertainty, and interpretation

For an RI, a **false positive** means the scanner reports a supported indicator that manual review finds unsupported under its defined conditions. A **false negative** means manual review identifies an in-scope supported indicator that the scanner did not report as supported. A missed additional command or rule match is not automatically a false-negative RI if the indicator category was already reported or the behavior falls outside its definition. Reviewers must also distinguish incomplete evidence from a confirmed absence of behavior.

The protocol limits indicator-accuracy calculations to comparisons assessable for the full scope: substantive relatedness, supported direction, adequately available evidence, and completed judgments on the eleven indicators. Other cases still inform error analysis and limitations. The sampled review strata intentionally include positive, negative, and difficult cases; their unweighted metrics cannot automatically be generalized to the entire GitSkills population.

**Manual validation is underway.** The saved three-trial summaries do not yet include completed independent and adjudicated accuracy or relationship-agreement results. We therefore do not report precision, recall, or agreement rates here. Once the team's adjudicated annotations are available, this section will be completed with those results, documented disagreements, and representative correctly and incorrectly classified cases. Appendix C describes the samples and Appendix E links their source artifacts.

## Discussion

### What the automated results suggest

The three Sprint 3 trials identify candidate introductions of security-sensitive behavior in **63 of 300 sampled families**. This provides an initial answer to the research question: the defined changes are observable in related agent-skill artifacts. It does not yet establish that the scanner is correct in every case, that the selected direction is the true history of copying, or that any skill is malicious. Independent human validation remains necessary.

The difference between family-level and pair-level rates is important. The family-level results are **25%, 18%, and 20%** across trials, whereas comparison-level results are **17.0%, 5.7%, and 22.2%**. A small number of families, especially `cso` and `mcp-builder`, generate many related comparisons. Counting every comparison equally can give those families much more influence than families with only one or two usable relationships. The sensitivity analysis in Section 7 shows that this is not a minor reporting choice. Family-level frequency, chosen in RDR-013, better describes how widely the automated findings appear across our **selected** families.

### Why the distinctions in the method matter

The scanner recognizes more changes than the indicator assessor supports: **1,119 comparisons were flagged**, but only **340** met the criteria for at least one supported RI. This is intentional. A changed package-manager command, for example, can be real evidence without establishing new external-code acquisition. Conversely, the `python-executor` case introduces an explicit remote download-and-execute operation even though network-dependent installation was already present in the base. Both examples illustrate why the result must describe *what changed*, rather than assuming every changed command increases risk.

The most widely observed indicators involve **local script execution** (**39 families**) and **external code or dependency acquisition** (**26 families**). These patterns are relevant to supply-chain review because they can change which code an agent is directed to run or obtain. Other indicators are concentrated: destructive or privileged file operations appear in only ten families but in 159 comparisons, many involving the same few families. We should not treat those repeated observations as independent security incidents.

### Alternative explanations and limits

Many changes may reflect intended features, routine maintenance, or a different environment rather than unwanted exposure. Larger families offer more opportunities to observe changes; some contain many similarity clusters and may group unrelated skills sharing a name. Some content differences are directed through chronology, while others rely on weaker structural containment. Missing supporting-file text can also hide or limit interpretation of script behavior.

These limitations do not erase the observed changes. They define what we can responsibly say about them. The contribution is a reproducible process for locating and explaining potentially consequential changes, along with direction and coverage evidence that a reviewer can inspect. Manual review will determine how reliably those candidates satisfy the study's definitions and may change the final interpretation.

## Threats to validity

**Construct validity.** The eleven RIs define selected observable security-sensitive changes, not every security risk. Regular-expression rules and evidence extraction can mistake examples, warnings, or equivalent commands for behavior changes and may miss unfamiliar syntax, indirect operations, or instructions expressed only in prose. A larger number of matches is not automatically a higher level of risk. A complete evidence record means a recognized construct was recovered; it does not establish the behavior's runtime effect, safety, or intent.

**Relatedness and historical order.** Identical names can group unrelated skills; renamed relatives can be missed. Token-shingle containment and Jaccard support relatedness but do not prove copying, and similarity clusters do not identify verified original authors or sources. GitSkills does not supply complete historical contents. Commit dates refer to observations at current file paths; equivalent-peer chronology, repository-creation boundaries, and containment extend coverage but remain indirect. **566 of 2,157** directed comparisons use containment as their basis, including **131 of 340** comparisons with supported RIs. Their inferred order needs particular caution.

**Missing evidence.** Some sibling files have no stored text, and historical enrichment is uneven. The three trials report **211 comparisons with incomplete evidence coverage**, including **81** that also have supported indicators. A supported observation can still have complete local evidence when another part of its comparison is missing. Conversely, complete-labelled comparisons can contain behavior outside the rules. Unknown sibling state is not equivalent to an empty or unchanged bundle.

**Sampling and generalizability.** All three trial samples were stratified at 20/55/20/4/1 families by artifact-count group and required at least one usable primary directed relationship. Larger families were deliberately oversampled; candidate families rejected by the selection screen were not counted as having no findings. The 300 sampled families are non-overlapping but not independent draws from a constant pool because later trials exclude prior selections. Neither the **21.0% family-level rate** nor the **15.8% comparison-level rate** estimates prevalence among all GitSkills families. Family size measures artifact occurrences, not independent authors or historical modifications.

**Dependence and conclusion validity.** Multiple directed comparisons may share artifacts or repeat the same underlying evidence. For example, `mcp-builder` accounts for 133 of the 191 supported-indicator comparisons in Trial 3. Pair-level counts are therefore not independent incidents. The study analyzes primary predecessor edges only; additional directed comparisons could contain distinct observations that are not included. Trial-to-trial comparisons and dominant-family exclusions show sensitivity, but do not substitute for a population-weighted study or a matched all-edge robustness analysis.

**Validation and reproducibility.** The three trials generated 90 directed and 45 relationship-review cases, but their human adjudication is in progress. Scanner precision, recall, and relationship agreement are not established by the automated summaries or by AI-assisted draft notes. The pipeline preserves manifests, data identities, code revision, rules, seeds, and case checksums; numeric artifact IDs remain local to a particular DuckDB build. The saved manifests still mark approval as provisional. Reproducibility safeguards limit accidental changes but do not eliminate selection bias or detection error.

The final conclusion must distinguish supported automated indicators from validated cases, and reported uncertainty from an actual absence of evidence. The full supporting dis-

cussion is maintained in `THREATS_TO_VALIDITY.md` and the RDRs.

## Conclusion

We studied whether reused or modified agent skills introduce security-sensitive behavior absent from a related earlier observed artifact. Our pipeline uses content similarity, explicit direction evidence, static evidence comparison, and eleven defined risk indicators to identify changes that warrant human review.

Across **three non-overlapping Sprint 3 trials**, the automated analysis found supported indicators in **63 of 300 sampled families** (**21.0%**) and **340 of 2,157 primary directed comparisons** (**15.8%**). Script-execution and external-code acquisition changes appeared across the most families. Pair-level percentages varied substantially because a few large or highly connected families contributed many repeated comparisons; the family-level result provides a more balanced view of the selected sample.

These are **candidate findings**, not confirmed vulnerabilities, evidence of malicious authorship, or population-wide prevalence estimates. The dataset does not provide verified copying histories, some evidence is incomplete, and manual evaluation of the similarity decisions and indicator judgments is underway. The final interpretation may change when those reviews are adjudicated.

The main contribution is a reproducible method that preserves what changed, the evidence for comparison direction, and the limits of what the scanner can establish. This makes detected differences useful for further review without presenting every changed command as a newly introduced security threat.

## References

Agent Skills. n.d. *Specification*. Accessed October 26, 2026. https://agentskills.io/specification.

Destefanis, Giuseppe, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2026. *GitSkills: A Dataset of Agent Skills on GitHub*. Version 1.0.0. Zenodo. https://doi.org/10.5281/zenodo.21875637.

## Appendix A Dataset construction and preparation

### Candidate selection

GitSkills is imported from the released SQLite database into a separate DuckDB database. The following query from `sql/create_analysis_tables.sql` materializes the comparison candidates without changing the source `artifacts` table.

```
CREATE TABLE artifact_groupings AS
WITH filtered AS (
    SELECT
        a.id AS artifact_id,
        a.repo_id,
        a.name,
        regexp_replace(
            lower(trim(a.description)),
            '\s+',
            ' ',
            'g'
        ) AS normalized_description
    FROM artifacts AS a
    WHERE a.frontmatter_valid = 1
      AND a.body_chars > 50
      AND a.content IS NOT NULL
      AND a.name IS NOT NULL
      AND a.description IS NOT NULL
),
eligible_names AS (
    SELECT
        name
    FROM filtered
    GROUP BY name
    HAVING COUNT(*) >= 2
),
groups AS (
    SELECT
        f.name,
        f.normalized_description,
        COUNT(*) AS artifact_group_size
    FROM filtered AS f
    JOIN eligible_names AS e ON e.name = f.name
    GROUP BY
        f.name,
        f.normalized_description
),
group_ids AS (
    SELECT
        row_number() OVER ()::BIGINT AS id,
        name,
        normalized_description,
        artifact_group_size
    FROM groups
)
SELECT
    g.id,
    f.artifact_id,
    f.repo_id,
    f.name,
    f.normalized_description,
    g.artifact_group_size,
    NULL::BIGINT AS sibling_file_count,
    NULL::VARCHAR AS sibling_content_sha
FROM filtered AS f
JOIN eligible_names AS e ON e.name = f.name
JOIN group_ids AS g ON g.name = f.name
 AND g.normalized_description =
  ↪ f.normalized_description;
```

Candidates require valid frontmatter, more than 50 body characters, available content/name/description, and at least two qualifying occurrences with the same exact name. These conditions select analyzable comparison candidates; they do not establish safety, reuse, or common ancestry. Short, malformed, metadata-poor, and singleton skills are excluded.

Descriptions are lowercased, trimmed, and whitespace-normalized for subgrouping. Family analysis compares across description subgroups within the same exact name, preserving candidates whose descriptions changed. The repeated-name requirement counts occurrences, including identical copies, rather than distinct content or repositories.

### Preparation workflow

The activity diagram connects candidate preparation to the end-to-end cohort analysis. Preparation counts come from the October 15, 2026 local audit; the three Sprint 3 samples are drawn from a separately screened subset of this candidate pool.

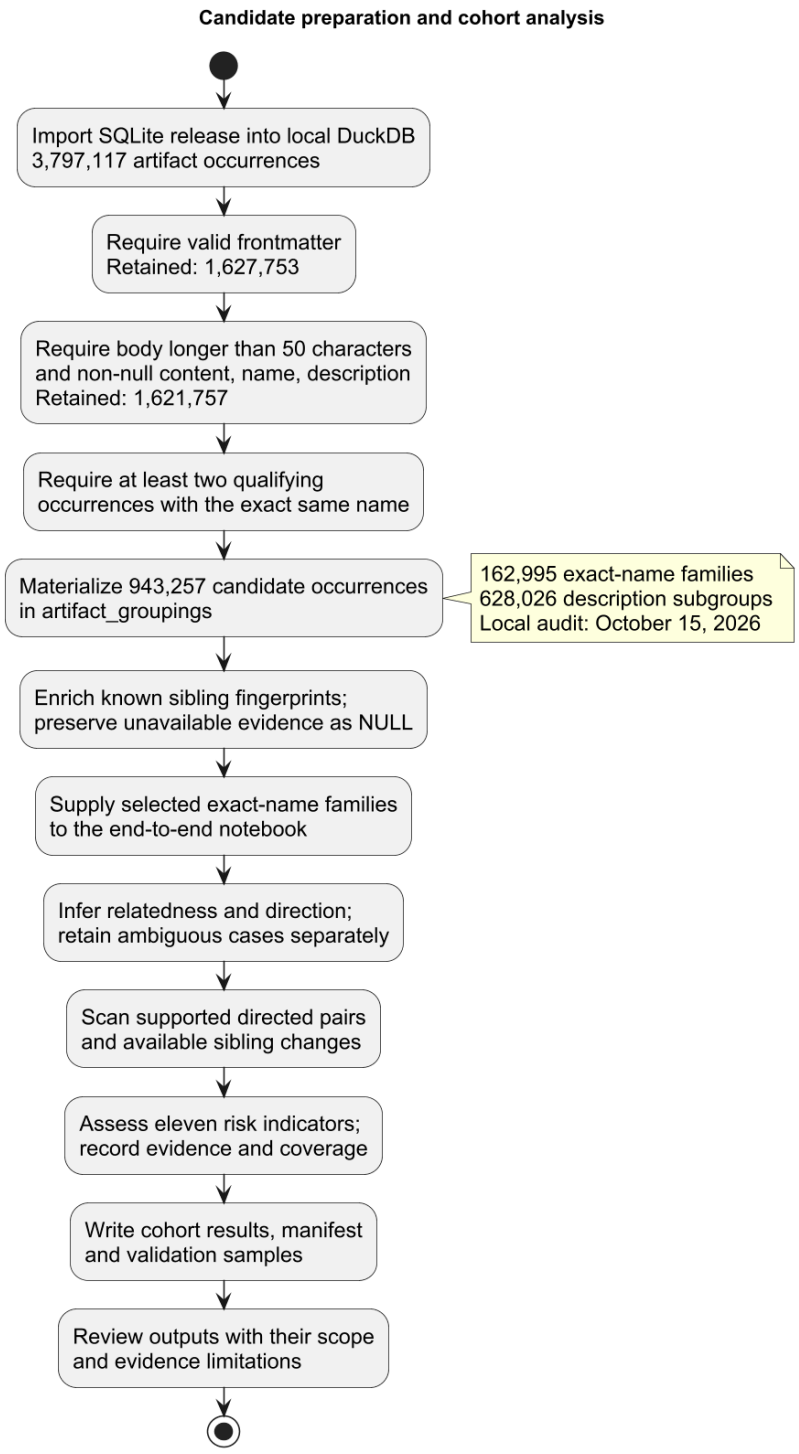



Figure 3: Candidate preparation and the end-to-end cohort analysis workflow.

| Cumulative selection stage | Retained artifacts | Excluded at this step |
|---|---|---|
| All source artifact occurrences | 3,797,117 | - |
| `frontmatter_valid = 1` | 1,627,753 | 2,169,364 |
| `body_chars > 50` | 1,621,757 | 5,996 |
| `content IS NOT NULL` | 1,621,757 | 0 |
| `name IS NOT NULL` | 1,621,757 | 0 |
| `description IS NOT NULL` | 1,621,757 | 0 |
| Exact name occurs in at least two qualifying rows | 943,257 | 678,500 |

Table 9: Candidate-selection audit for the prepared GitSkills snapshot

### Selection audit and identity

The audit retained 943,257 occurrences in 162,995 exact-name families. These are **not** the final family-selection pool or results denominator. The Sprint 3 family picker excluded names unsuitable for one-name-per-line selection, screened distinct main-file/sibling hash states, and checked for at least one usable primary directed comparison. Its sampling reports list 162,461 candidates at the preliminary selection stage; **not all of these candidates pass relationship validation**. The final analysis covers 300 randomly selected, validated families and 6,570 loaded artifact occurrences across three non-overlapping trials.

Artifact and description-group numbers are local to the prepared database. The query assigns group numbers without an explicit ordering, so cross-build reconciliation must use the dataset snapshot, repository/path identity, and content hashes. The sibling fields begin as null placeholders; subsequent enrichment distinguishes known empty bundles from unavailable or incomplete sibling evidence. Null must not be interpreted as an empty bundle.

### Sprint 3 sample audit

The following counts come from the saved `sampling_report.json` files. A rejection means that the candidate failed the family picker's relationship validation, **not** that it contained no security-sensitive behavior. The picker replaced rejected candidates in the same stratum.

Each trial retains the same 20/55/20/4/1 size allocation described in Section 5. The sampling reports preserve the rejection reasons, source database identity, thresholds, excluded `families.txt` paths and checksums, and exact selected names. The 162,461 preliminary candidates are **not** an eligible-population denominator for estimating risk prevalence, because a validated primary directed relationship is required to enter the study.

## Appendix B Pipeline implementation

### Detection rule registry

The saved Sprint 3 manifests identify **15 active rules across six categories**. These rules recognize candidate evidence, while the eleven risk indicators (RIs) in Section 6 assess whether *changes* in that evidence satisfy specific criteria. A new rule match, increased rule count, or changed command text alone does not establish an RI.

The executable definitions are in `src/gitskills/rules/registry.py` and the category modules. In particular, RDR-012 distinguishes explicit package installation (`EXT-002`) from manifest-based dependency installation (`EXT-004`), while `EXT-003` detects staged downloaded-file execution. Overlapping rules can describe the same command, so individual matches are not independent events. Leading `SKILL.md` frontmatter is masked for behavioral analysis; sibling-file content is scanned separately when eligible and available. Dataset instructions are never executed.

### End-to-end workflow and saved policy

For each selected family, the pipeline loads all qualifying artifacts, forms content variants and similarity clusters, evaluates the direction hierarchy, selects primary predecessors, and scans the directed pairs. It extracts evidence from both skill bodies and eligible added or changed sibling files, compares the base and derived evidence, applies RI-001 through RI-011, and records uncertainty and missing content. Cached family scans are reused only when their source, implementation, rule, and policy fingerprints are compatible.

Direction priority is declared source, direct chronology, equivalent-peer chronology, repository-creation boundary, then asymmetric containment. These are inference rules, not proofs of copying. Additional directed edges exist in the analysis method but **were not scanned for the three Sprint 3 result sets**. Ambiguous directions and equivalent peers remain available for relationship review without being treated as evidence that a behavior was introduced.

### Main-file and sibling-change classification

Similarity equivalence does not require the same byte hash, so a `siblings-only` comparison can still contain textual differences in `SKILL.md`. Added or changed sibling **files** with available nonempty content are eligible for static scanning, regardless of extension. Unavailable content is reported as missing; paths, filenames, and directory changes alone are not treated as the behavior of an unknown script.

### Relationship and scanning diagrams

The first diagram shows how the analyzer separates relatedness, direction, equivalent peers, and selected versus additional edges. It is a **conceptual diagram**; only selected primary edges enter the Sprint 3 security counts. The second diagram outlines the family-scanning component, including conditional sibling analysis. Risk-indicator assessment and cohort aggregation follow the underlying evidence comparison.

| Sample | Seed | Families selected | Candidates rejected during drawing | Previously selected families excluded |
|---|---|---|---|---|
| Trial 1 | 5801 | 100 | 239 | 0 |
| Trial 2 | 5802 | 100 | 288 | 100 |
| Trial 3 | 5803 | 100 | 247 | 200 |

Table 10: Sprint 3 sample seeds and family-selection audit

| Category | Active rules | What they recognize | Main limitation |
|---|---|---|---|
| Command execution | `CMD-001`, `CMD-002` | Shell-command blocks and recognized command invocations | A code example is not proof of execution. |
| Network access | `NET-001`, `NET-002` | HTTP(S) URLs and recognized network clients | A documentation link alone is not a network operation. |
| File-system access | `FS-001`, `FS-002`, `FS-003` | File commands, writing APIs, and reading commands/APIs | Ordinary file use is not automatically sensitive or destructive. |
| Credentials | `CRED-001`, `CRED-002` | Secret-related references and recognized credential files | A name or placeholder does not prove access to a secret. |
| External code and dependency acquisition | `EXT-001`, `EXT-002`, `EXT-003`, `EXT-004` | Direct download-to-shell, explicit package installs, staged download-and-execute, and manifest-based dependency installs | A legitimate installation may match; multi-step behavior may be incomplete. |
| System modification | `SYS-001`, `SYS-002` | Privilege elevation and system-changing commands | Administrative instructions can be legitimate. |

Table 11: Fifteen detection rules and their coverage limits

| Setting | Sprint 3 value |
|---|---|
| Main study | Three non-overlapping, 100-family stratified samples |
| Family seeds | 5801, 5802, 5803 |
| Manual review seed | 580 |
| Edge selection | `primary` only |
| Shingle width | 5 tokens |
| Minimum containment | 0.80 |
| Minimum Jaccard | 0.30 |
| Minimum shared shingles | 5 |
| Containment direction margin | 0.10 |
| Detection rules / risk indicators | 15 / 11 |
| Source code revision | `ae69732ed8988ad7066ff351848f7d2e8ad311a1` |

Table 12: Pinned analysis configuration for Sprint 3

| Change type | Meaning |
|---|---|
| `skill-only` | Main skill text differs; known sibling state is equivalent. |
| `siblings-only` | Main skill text is equivalent **under the analyzer's similarity representation**; known sibling state differs. |
| `skill+siblings` | Main skill text and known sibling content both differ. |
| `equivalent` | Both main-file and known sibling states are equivalent. |
| `unknown` | Missing or uncertain sibling state prevents reliable bundle classification. |

Table 13: Bundle change classifications

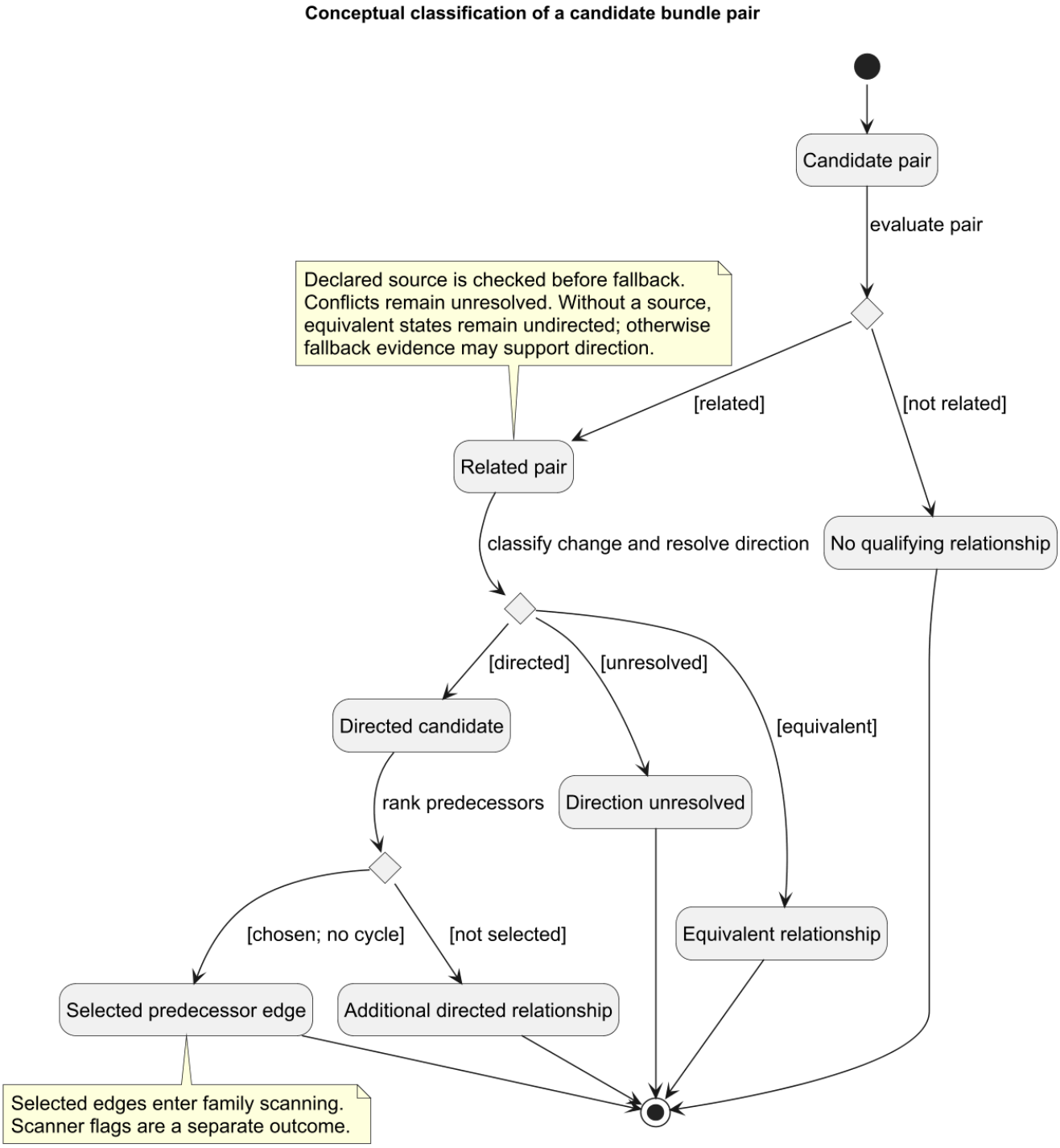



Figure 4: Conceptual UML state diagram for relationship classification. Scanner flags are separate from lineage outcomes.

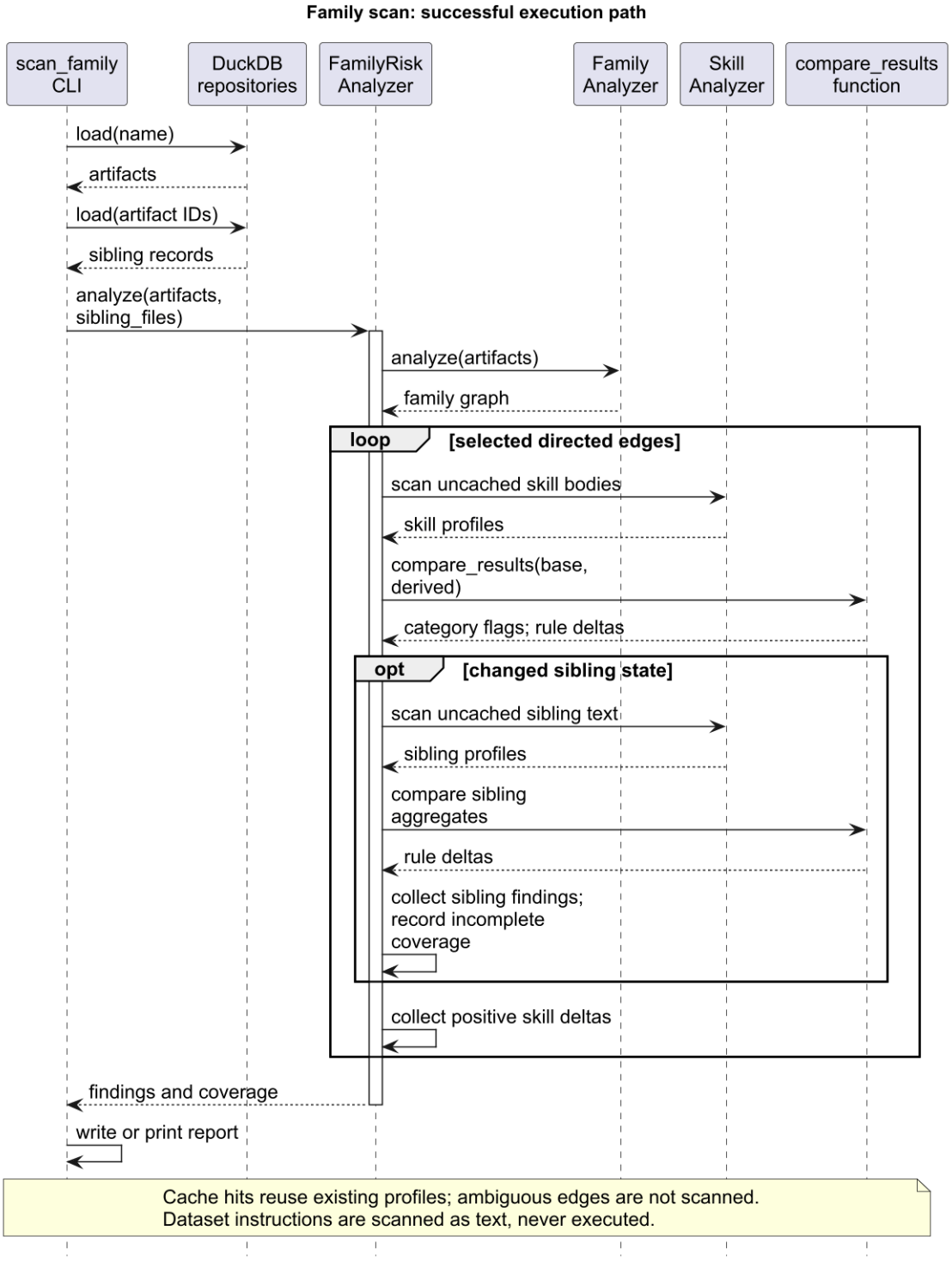



Figure 5: Family scanning and conditional sibling analysis; shown as a component-level summary rather than the complete research aggregation.

## Appendix C Validation scope and status

Each of the **three Sprint 3 trials** generated **30 directed-comparison review cases** (five from each of six strata) and **15 relationship-review cases** (five from each of three strata), using review seed **580**. The total is **90 directed cases and 45 relationship cases**. These samples come from the saved results for each separate 100-family trial; they are **not** additional data collected for the final analysis.

Directed review covers supported indicators, uncertain indicators, flagged comparisons without supported indicators, coverage-limited comparisons, sibling changes, and unflagged directed comparisons. Relationship review includes equivalent, ambiguous, and no-related-edge cases. The two reviews serve different purposes: relationship review examines the similarity and ordering decisions, while directed review examines extraction, source differences, RI judgments, and coverage.

Reviewers distinguish rule matches, extracted evidence, supported versus uncertain indicator decisions, missing evidence, and evidence for direction. A reported RI is a **false positive** when manual inspection does not support it under the defined indicator policy. A **false negative** requires a manually supported *in-scope* indicator that the scanner did not report as supported. A reviewer should not count a benign change outside RI-001 through RI-011 as a missed indicator merely because it is security-related in another sense.

At the time of the saved runs, the annotation templates and selected cases existed, but **no independently adjudicated Sprint 3 validation metrics were included** in the summaries. Manual review is ongoing. We therefore do not report precision, recall, reviewer agreement, or confirmed positive/negative counts yet. When adjudicated files are available, the pipeline can recompute the validation portion of each saved summary without changing the original sample or scanner outputs.

The stratified review samples help characterize error types and coverage gaps, but their raw proportions should not be treated as population-wide accuracy estimates without a justified sampling calculation. Detailed review instructions are in `docs/instructions/manual_annotation_protocol.md`; the saved case lists and manifest checksums are linked in Appendix E.

## Appendix D Additional Sprint 3 results

The tables below are calculated from the saved Sprint 3 `review/summary.json`, `review/families.csv`, `review/directed_pairs.csv`, and `review/direction_basis.csv` files. All indicator results are **automated** and pending independent human adjudication. Per-indicator counts overlap: one comparison may contain more than one supported indicator, and multiple comparisons may share artifacts.

### Indicator frequency by trial

Each entry counts **primary directed comparisons** with the indicated supported RI. The final column counts **distinct families** across the three non-overlapping trials that have at least one such comparison.

### Direction evidence and selected comparisons

The following categories are mutually exclusive per directed comparison. Counts of comparisons with RIs are *not* counts of verified copied-skill lineages.

### Bundle change types

`unknown` describes an unavailable or insufficiently known sibling state, not a known unchanged bundle. Equivalent relationships are not counted as base-to-derived comparisons. A `siblings-only` classification uses main-skill equivalence under the similarity representation rather than requiring identical byte hashes; see Appendix B.

### Family-size and cluster notes

Across 300 families, the analyzer produced **438 similarity clusters**. Families with multiple clusters were uncommon in the smaller strata (0 of 60 Low families; 10 of 165 Medium) and more common in the larger strata (19 of 60 Large; all 12 X-Large; all three Extreme). These clusters indicate relatedness under the project's token-shingle method, not verified independent origins.

For illustration, selected large families show different cluster structures:

The full per-family counts, coverage records, and selected comparisons remain in the three saved `review/` directories. They are not duplicated here because the detailed tables are best inspected directly as reproducible data files.

## Appendix E Supporting repository material

The report uses the **three saved Sprint 3 trials**, not the earlier Sprint 2 exploratory cohort. The source code revision recorded in all three manifests is `ae69732ed8988ad7066ff351848f7d2e8ad311a1`; the manifests additionally retain the implementation fingerprint, active rules, dataset identity, and selected inputs. Results are not recomputed by rebuilding the report.

The saved research inputs and outputs are organized as follows (each trial has the same layout):

```
notebooks/end_to_end_pipeline/sprint3/
  samples1/families.txt
  samples1/sampling_report.json
  samples2/families.txt
  samples2/sampling_report.json
  samples3/families.txt
  samples3/sampling_report.json
  results1/
    sample_manifest.json
    execution_summary.json
    review/summary.json
    review/families.csv
    review/directed_pairs.csv
    review/indicator_frequency.csv
    review/direction_basis.csv
    manual-review/directed-pairs/
    manual-review/relationship-pairs/
  results2/                  # same structure
  results3/                  # same structure
```

| Indicator | Trial 1 | Trial 2 | Trial 3 | All comparisons | Distinct families |
|---|---|---|---|---|---|
| **RI-001** | 0 | 0 | 3 | 3 | 3 |
| **RI-002** | 0 | 0 | 0 | 0 | 0 |
| **RI-003** | 8 | 3 | 41 | 52 | 17 |
| **RI-004** | 0 | 0 | 0 | 0 | 0 |
| **RI-005** | 25 | 15 | 115 | 155 | 26 |
| **RI-006** | 57 | 7 | 95 | 159 | 10 |
| **RI-007** | 3 | 0 | 1 | 4 | 2 |
| **RI-008** | 8 | 4 | 6 | 18 | 8 |
| **RI-009** | 0 | 0 | 0 | 0 | 0 |
| **RI-010** | 33 | 23 | 145 | 201 | 39 |
| **RI-011** | 0 | 0 | 1 | 1 | 1 |

Table 14: Risk-indicator frequency across the three trials

| Direction basis | All comparisons | With supported RIs | Incomplete evidence coverage |
|---|---|---|---|
| Declared source | 10 | 0 | 1 |
| Direct chronology | 399 | 55 | 42 |
| Equivalent-peer chronology | 355 | 43 | 8 |
| Repository-creation boundary | 827 | 111 | 86 |
| Directional containment | 566 | 131 | 74 |
| **Total** | **2,157** | **340** | **211** |

Table 15: Direction evidence and coverage for selected comparisons

| Change type | Comparisons | With supported RIs |
|---|---|---|
| `skill-only` | 805 | 33 |
| `siblings-only` | 190 | 41 |
| `skill+siblings` | 804 | 242 |
| `unknown` | 358 | 24 |
| **Total** | **2,157** | **340** |

Table 16: Primary-comparison counts by bundle change type

| Family | Artifact occurrences | Similarity clusters |
|---|---|---|
| `implement` | 856 | 35 |
| `mcp-builder` | 543 | 11 |
| `documentation` | 508 | 19 |
| `cso` | 150 | 2 |

Table 17: Examples of family size and similarity clusters

| Repository resource | Purpose |
|---|---|
| End-to-end notebook | Reproduces cohort processing and result tables. |
| Pipeline README | Configuration, outputs, cache, and validation instructions. |
| Dataset preparation | GitSkills acquisition and DuckDB preparation. |
| Candidate-selection SQL | Generates the comparison candidate pool (Appendix A). |
| Sibling fingerprint SQL | Represents known versus unknown bundled-file content. |
| Sampling decision RDR-014 | Family eligibility and stratified sampling. |
| Edge-selection decision RDR-013 | Rationale for primary-only comparisons and family-level reporting. |
| Risk indicator definitions RDR-010 | Operational scope of RI-001 through RI-011. |
| Threat model | Trust boundary, detection scope, and security interpretation limits. |
| Annotation protocol | Independent review and adjudication guidance. |
| Report build guide | Source sections, figures, and local report rendering. |

Table 18: Reproducibility and supporting repository resources

The `sampling_report.json` files record sampling seeds **5801–5803**, stratum allocations, exclusions, and candidate rejections. The `sample_manifest.json` files record the selected families, policies, scanner and code identities, review sample checksums, and a source fingerprint. Research summaries and CSV files contain the automated outcome counts used in Sections 5–7 and Appendix D; audit and execution files provide coverage and processing status. The separate annotation files are reviewer-owned and remain distinct from automated predictions.

Two cross-trial charts in Section 7 are generated from the saved `families.csv` and `directed_pairs.csv` files using `report/figures/generate_sprint3_figures.py`. The script does not alter trial results or select new families.

The three sample manifests currently mark their approval reference as **provisional** (`sample_status: provisional`, no approval reference). The final report should not imply formal approval has been recorded unless the manifest or repository decision history is updated. A tagged release and final manual-validation record should be linked when available.

## Appendix F Use of AI in development and report preparation

Project Group 8 used AI assistance during tool and pipeline development and in preparing this report. Development assistance included discussing implementation approaches, drafting and revising code, troubleshooting errors, and supporting documentation and verification work. These activities contributed to the components integrated into the end-to-end pipeline.

For the report, AI assistance supported organizing material, drafting and revising prose, condensing earlier documentation, and checking consistency between the narrative and saved outputs. It also supported formatting and rebuilding the report. The numerical findings come from the saved pipeline artifacts; AI-generated explanations are not experimental measurements.

Project Group 8 remains responsible for the implementation, interpretation, and final submitted text. Generated code and prose require checking against project requirements, source evidence, and reproducible outputs. AI assistance does not establish the correctness of inferred relationships or reported indicators, and it does not resolve the validation limitations stated in the main report.